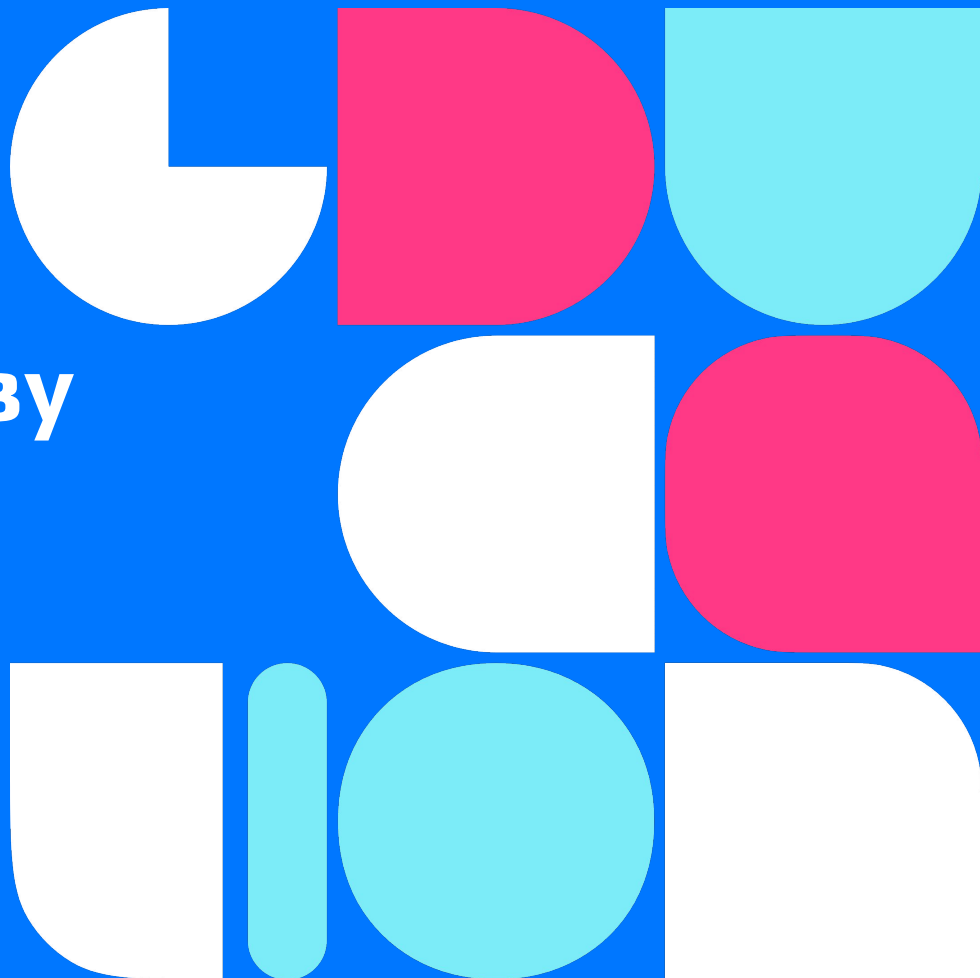
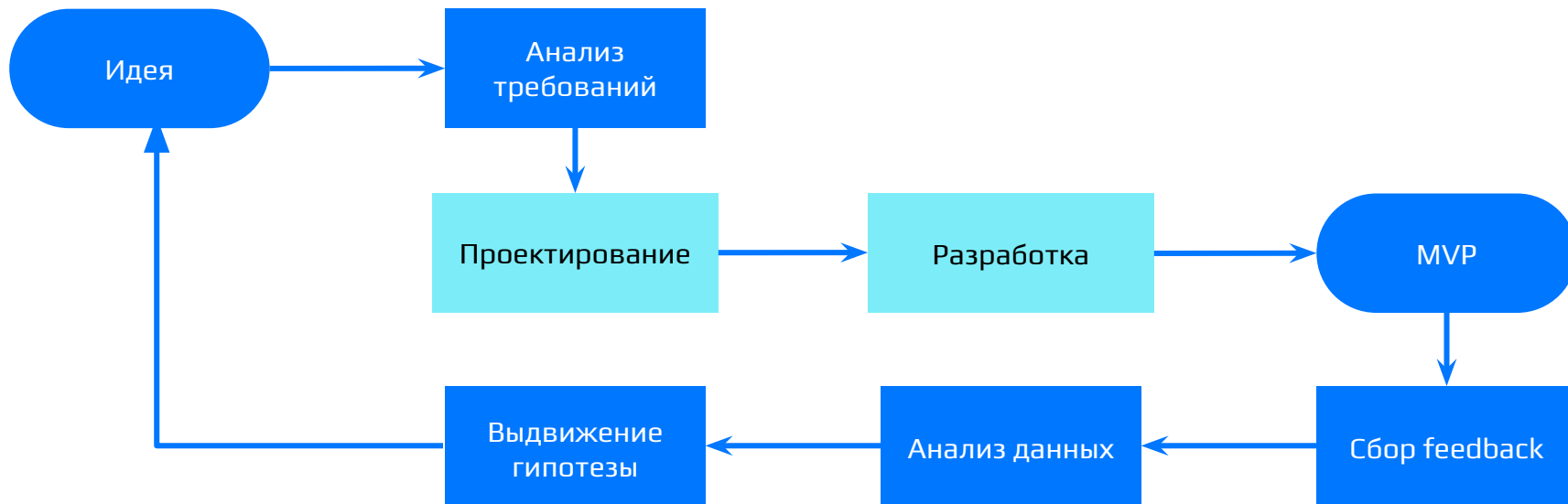




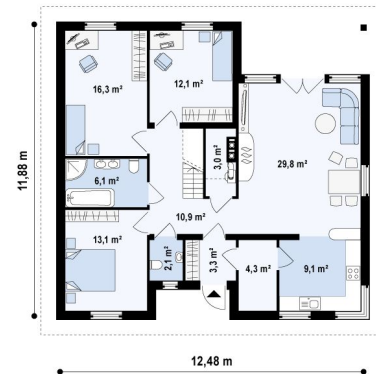
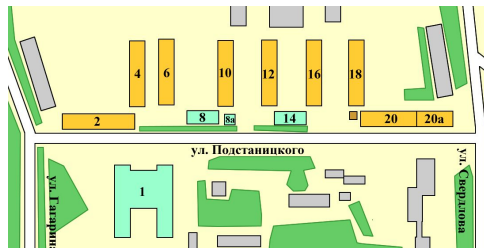
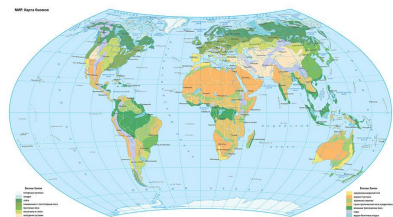
Олег Гибадулин

Этапы



C4

Архитектура как Google Maps

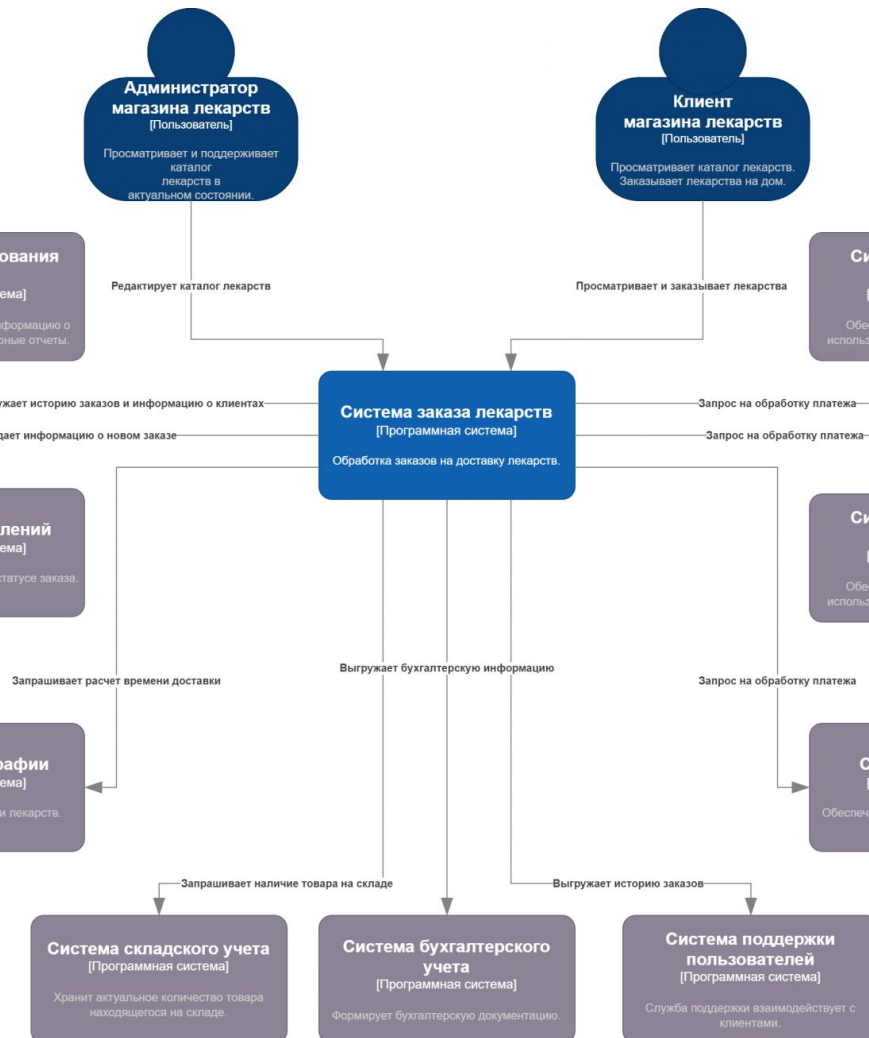
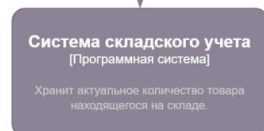
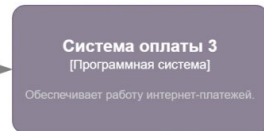
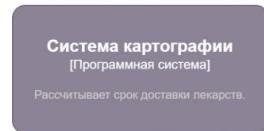
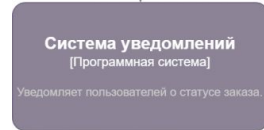
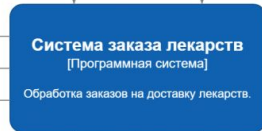
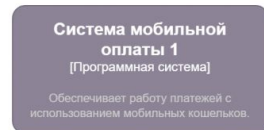
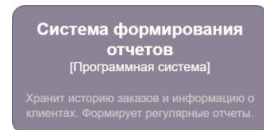


Уровни

- **Уровень 1: Context (Контекст).** Система целиком и ее связь с внешним миром (пользователи, чужие API).
- **Уровень 2: Containers (Контейнеры).** Разделение на физические приложения (iOS, Сервер, База Данных).
- **Уровень 3: Components (Компоненты).** Внутреннее устройство одного контейнера (контроллеры, сервисы).
- **Уровень 4: Code (Код).** Классы, функции и интерфейсы.

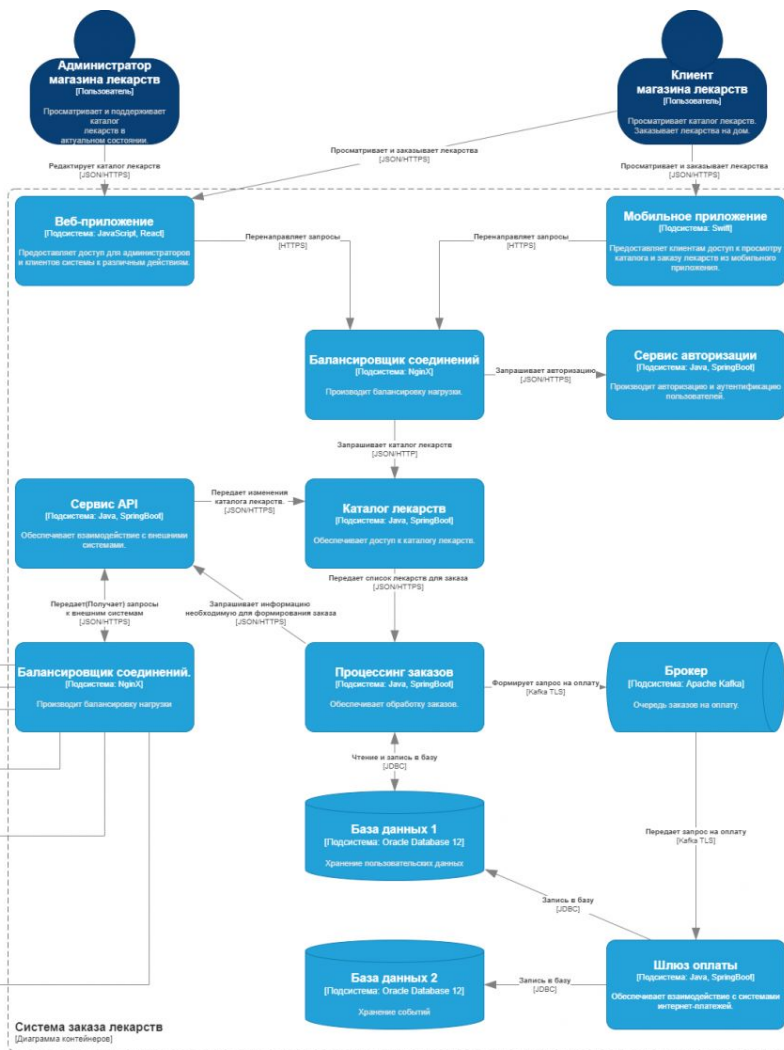
Уровень 1. Контекст Системы

- **Фокус на бизнес-ценности:** Схема должна быть понятна инвестору и нетехническому менеджеру.
- **Никаких технологий:** Мы не пишем тут про базы данных, языки программирования и серверы.
- **Границы ответственности:** Всё, что внутри синего квадрата - мы пишем сами. Серые квадраты - внешние интеграции, которые мы не контролируем.



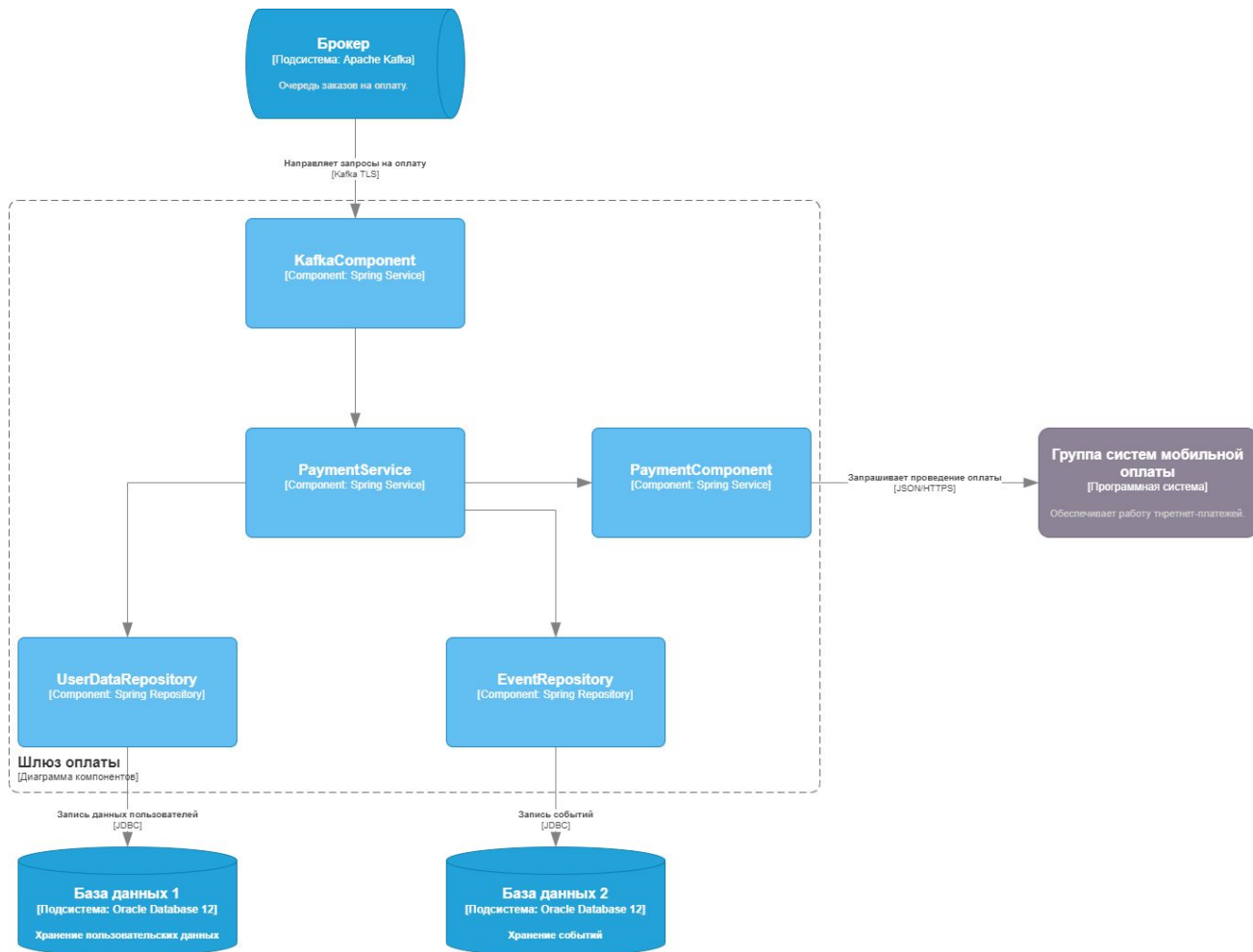
Уровень 2. Контейнеры

- **Контейнер** - это то, что можно запустить (Приложение, Сервер, База данных).
- **Разделение труда:** Nginx держит удар, Python думает, PostgreSQL хранит логику, S3 хранит файлы.
- **Стек технологий:** Выбирается под задачу бизнеса (MVP = скорость разработки $\rightarrow$ Python).



Уровень 3. Компоненты

- **Компонент** - это набор классов/функций, объединенных одной задачей.
- **Структура папок:** Компоненты на схеме СЗ часто совпадают со структурой папок в вашем проекте.



Уровень 4. Код

Используется для документирования структуры кода. На практике не используется, т.к. разработчикам это не надо. Это как диаграмма классов **UML**: вроде полезно понимать, а по факту не нужна.

Практика

`python manage.py runserver`



Gunicorn

- Менеджер процессов
- Он клонирует ваше приложение. Формула: $\text{CPU} \times 2$.
- Сервер на 4 ядра = 8 воркеров = 8 одновременных процессов.
- Математика: 8 параллельных запросов по 300 мс = неплохая нагрузка (десятки RPS), но всё ещё имеет потолок.

Golang

- 1 инстанс Go = 10 000+ одновременных запросов.
- Работает на Горутинах (сверх-легковесных потоках внутри процесса). Ему не нужен Unicorn

nginx

- Фильтрует опасные запросы и сетевой мусор
- Удобная настройка HTTPS
- Быстро отдает статику
- Решает проблему медленного клиента
- Кеширует ответы

Работа с файлами

- Локально + Nginx
- S3 Object Storage

Платежный треугольник

- **Polling:** Мобилка каждые 3 секунды спрашивает Бэкенд: "Статус изменился?".
- **Deep Links:** Банк сам перекидывает пользователя обратно в приложение и приложение делает 1 запрос в API
- **WebSockets:** Сервер сам "пушит" сигнал в мобилку по открытому TCP-каналу

Асинхронность и Очереди

- Асинхронность в памяти (Go Goroutines / FastAPI BackgroundTasks)
- Очередь в Базе Данных (PostgreSQL)
- In-Memory Message Broker (Redis + Celery)
- Advanced Brokers / Event Bus (RabbitMQ / Kafka)

Практическое задание №4

- **Схема C4:** Нарисуйте 1 и 2 уровни (Контекст и Контейнеры).
- **Стек:** Выберите и кратко обоснуйте выбор ключевых технологий для вашего MVP.

Спасибо за внимание!

Не забудьте оставить отзыв :)

